

GeoFare – 15 Principles of Software Engineering Analysis

Based on Davis's Early 15 Principles and the GeoFare source code supplied for review

Purpose and scope

This report evaluates the submitted GeoFare Android/Kotlin project against Davis's Early 15 Principles of Software Engineering. The assessment is based on the source files provided in this activity. **Not Demonstrated** means the supplied code does not provide enough evidence to verify the practice; it does not necessarily mean the team never performed it.

Status: Implemented = clear evidence; Partially Implemented = some evidence but gaps remain; Not Demonstrated = insufficient evidence in the submitted code.

Summary

#	Principle	Status
1	Make quality number 1	Partially Implemented
2	High-quality software is possible	Partially Implemented
3	Give products to customers early	Implemented
4	Determine the problem before writing the requirements	Partially Implemented
5	Evaluate design alternatives	Not Demonstrated
6	Use an appropriate process model	Not Demonstrated
7	Use different languages for different phases	Implemented
8	Minimize intellectual distances	Implemented
9	Put techniques before tools	Partially Implemented
10	Get it right before you make it faster	Partially Implemented
11	Inspect code	Partially Implemented
12	Good management is more important than good technology	Not Demonstrated
13	People are the key to success	Partially Implemented
14	Follow with care	Partially Implemented
15	Take responsibility	Partially Implemented

1. Make quality number 1

Status: Partially Implemented

Evidence from GeoFare: Validation, loading states, Firebase authentication, Firestore storage, navigation, and user-facing errors show attention to quality.

Gap / code that does not fully implement it: Several core screens remain placeholders, including fare calculation, route monitoring, trip history, and report concern.

Relevant files: LoginScreen.kt; RegisterScreen.kt; HomeScreen.kt; FareCalculationScreen.kt

2. High-quality software is possible

Status: Partially Implemented

Evidence from GeoFare: The project uses structured Compose screens, Material 3, Firebase, and navigation.

Gap / code that does not fully implement it: No automated tests or complete implementations of core features were supplied, so overall quality cannot be verified.

Relevant files: MainActivity.kt; HomeScreen.kt; LoginScreen.kt; RegisterScreen.kt

3. Give products to customers early

Status: Implemented

Evidence from GeoFare: The project is already organized as a runnable Android application with multiple screens and a shared GitHub repository.

Gap / code that does not fully implement it: The source does not document the project's release or user-feedback history.

Relevant files: MainActivity.kt; README.md; HomeScreen.kt

4. Determine the problem before writing the requirements

Status: Partially Implemented

Evidence from GeoFare: Screen functions clearly match GeoFare goals: fare calculation, route monitoring, trip history, and reporting concerns.

Gap / code that does not fully implement it: The original requirements analysis is not present in the supplied Kotlin source.

Relevant files: HomeScreen.kt; FareCalculationScreen.kt; TripMonitoringScreen.kt; TripHistoryScreen.kt; ReportConcernScreen.kt

5. Evaluate design alternatives

Status: Not Demonstrated

Evidence from GeoFare: The app has a clear Compose/navigation structure.

Gap / code that does not fully implement it: No alternative designs, architecture comparisons, or decision records were supplied. **Relevant files:** MainActivity.kt; HomeScreen.kt

6. Use an appropriate process model

Status: Not Demonstrated

Evidence from GeoFare: The repository shows incremental project work and commits.

Gap / code that does not fully implement it: The code does not prove whether Agile, Waterfall, iterative, or another formal process model was used.

Relevant files: GitHub repository; README.md

7. Use different languages for different phases

Status: Implemented

Evidence from GeoFare: Kotlin is used for Android source while Gradle Kotlin DSL is used for build configuration.

Gap / code that does not fully implement it: No other phase-specific language is evident in the supplied source, but the principle does not require unnecessary languages.

Relevant files: *.kt; build.gradle.kts; settings.gradle.kts

8. Minimize intellectual distances

Status: Implemented

Evidence from GeoFare: Names such as FareCalculationScreen, TripMonitoringScreen, fullName, userType, and createdAt closely match the application domain.

Gap / code that does not fully implement it: Some implementation decisions are lightly documented.

Relevant files: HomeScreen.kt; RegisterScreen.kt; MainActivity.kt

9. Put techniques before tools

Status: Partially Implemented

Evidence from GeoFare: The code applies validation, state management, navigation, and asynchronous Firebase operations.

Gap / code that does not fully implement it: The source does not document formal evaluation of alternative tools before selecting Compose/Firebase.

Relevant files: LoginScreen.kt; RegisterScreen.kt; HomeScreen.kt

10. Get it right before you make it faster

Status: Partially Implemented

Evidence from GeoFare: Input validation and loading states reduce invalid operations and repeated actions.

Gap / code that does not fully implement it: No profiling or benchmark evidence was supplied, and several core screens are unfinished.

Relevant files: LoginScreen.kt; RegisterScreen.kt; FareCalculationScreen.kt

11. Inspect code

Status: Partially Implemented

Evidence from GeoFare: The source is separated into screen files and uses consistent naming and structure.

Gap / code that does not fully implement it: No formal peer-review record, inspection checklist, pull request review, or static-analysis report was supplied.

Relevant files: All supplied Kotlin files; GitHub repository

12. Good management is more important than good technology

Status: Not Demonstrated

Evidence from GeoFare: The shared repository and README support coordination.

Gap / code that does not fully implement it: Project management, scheduling, leadership, risk management, and task assignment cannot be verified from source code.

Relevant files: README.md; GitHub repository

13. People are the key to success

Status: Partially Implemented

Evidence from GeoFare: A shared GitHub repository and documented Android structure allow group members to collaborate.

Gap / code that does not fully implement it: The source cannot prove communication, responsibilities, or team effectiveness.

Relevant files: GitHub repository; README.md

14. Follow with care

Status: Partially Implemented

Evidence from GeoFare: The project uses established Android/Jetpack and Firebase technologies appropriate to the application.

Gap / code that does not fully implement it: No experiments or formal technology-selection criteria were supplied.

Relevant files: MainActivity.kt; LoginScreen.kt; RegisterScreen.kt

15. Take responsibility

Status: Partially Implemented

Evidence from GeoFare: The code validates common user input and reports Firebase failures rather than silently ignoring them.

Gap / code that does not fully implement it: More safeguards and testing are needed, and several placeholder features remain incomplete.

Relevant files: LoginScreen.kt; RegisterScreen.kt; HomeScreen.kt

Overall Findings

The strongest evidence of software-engineering quality in the submitted GeoFare code is found in input validation, authentication handling, state management, navigation, domain-aligned naming, and user-facing failure messages. LoginScreen.kt and RegisterScreen.kt provide particularly clear examples.

Several Davis principles concern development activities rather than individual code statements. Requirements analysis, design-alternative evaluation, formal process-model use, management practices, peer inspection, and technology-selection experiments cannot be reliably proven from the supplied Kotlin files alone. These findings are therefore marked Not Demonstrated or Partially Implemented.

The project also contains unfinished placeholder screens. These are important non-implementation findings because they show that some advertised features are not yet functional in the supplied code.

Reference basis

Davis's Early 15 principles are commonly presented as: make quality number 1; high-quality software is possible; give products to customers early; determine the problem before writing requirements; evaluate design alternatives; use an appropriate process model; use different languages for different phases; minimize intellectual distances; put techniques before tools; get it right before making it faster; inspect code; good management is more important than good technology; people are the key to success; follow with care; and take responsibility.

Prepared from the GeoFare source code supplied by the project team.